SIMATS ENGINEERING

ASSESSMENT TOOL

COURSE NAME: Theory of Computation

COURSE CODE: CSA13

COURSE OUTCOME COVERED

CO1: Analyze fundamental mathematical concepts of automata theory for formal languages and decision problems(BL:3)

Assessment Tool Weightage: 20%

QUESTIONS: 5 **Total Marks:25**

Duration : 1 Hour

Simulation

Code of Conduct:

I **J. SAI DHEERAJ (192311191)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **J. Sai Dheeraj**

SIMATS ENGINEERING

ASSESSMENT TOOL

[1] Design a Non-Deterministic Finite Automaton (NBA) for an Automatic Door System based on sensor inputs.An automatic door system can be modeled using an NBA to represent different states and transitions. The system consists of four states:

Closed, Opening, Open, and Closing. The input symbols are P (Person detected) and N (No person detected). Initially, the door is in the Closed state. When a person is detected (P), the system may non-deterministically transition from Closed to Opening. Once opened, the system moves to the Open state. If no person is detected (N), the door transitions to Closing and eventually returns to the Closed state. Non- determinism occurs when the system may remain in the Open state even if a person is detected or may start closing if no detection occurs.

Concept

An NFA allows a state to have several possible next states for the same input (or none). The door controller is non-deterministic because, on P in Closed, it may stay Closed or begin Opening, and on N in Open it may stay Open or begin Closing. The automaton is written as $M = (Q, \Sigma, \delta, q_0, F)$.

Formal Definition

- $Q = \{Closed, Opening, Open, Closing\}$
- $\Sigma = \{P, N\}$
- Start state q_0 = Closed
- $F = \{Closed\}$ (a complete door cycle ends with the door back in the Closed state)
- δ is given by the transition table below.

State	P (Person)	N (No person)
→ * Closed	{Closed, Opening}	{Closed}
Opening	{Open}	{Open}
Open	{Open}	{Open, Closing}
Closing	{Opening} (safety re-open)	{Closed}

Transition Diagram

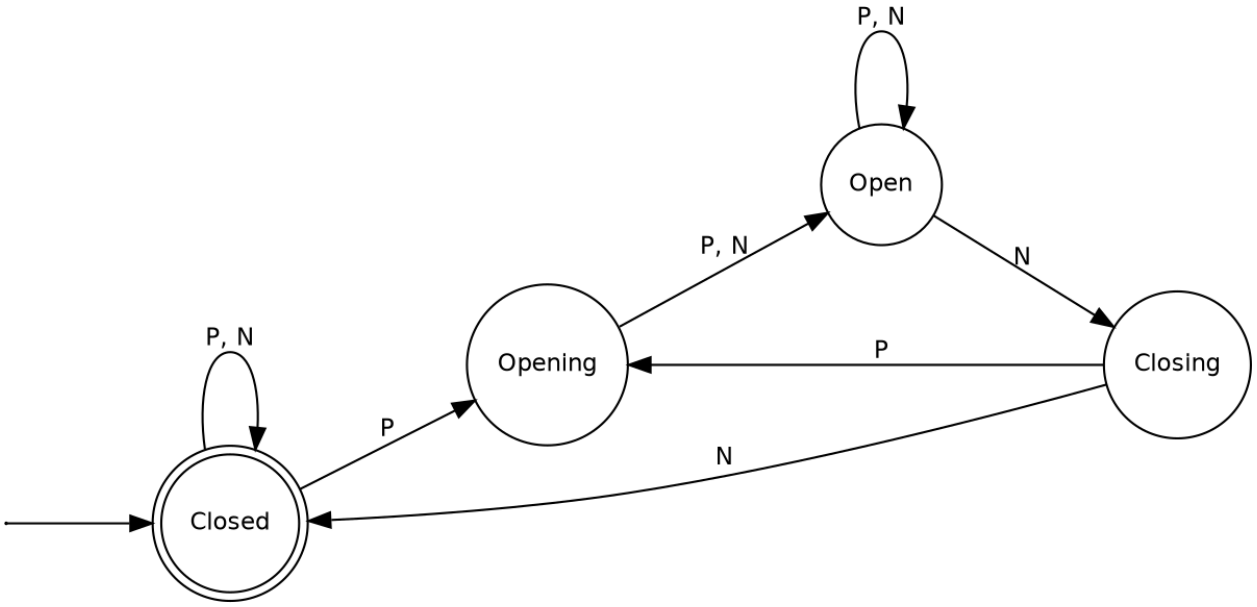


Fig 1 NFA for the automatic door system

Simulation (input string P P N N N)

The simulator follows all possible paths in parallel (the set of current states):

Step	Symbol read	Set of current states
0	—	{Closed}
1	P	{Closed, Opening}
2	P	{Closed, Opening, Open} (Closed on P → Closed/Opening; Opening on P → Open)
3	N	{Closed, Open, Closing}
4	N	{Closed, Open, Closing}
5	N	{Closed, Open, Closing}

Accepting path: Closed –P→ Opening –P→ Open –N→ Closing –N→ Closed. The path ends in Closed
∈ F, so the string is accepted: a person was detected, the door opened, and after no person was detected it closed again.

Result: The NFA models the door cycle Closed → Opening → Open → Closing → Closed with non-determinism at Closed (on P) and at Open (on N).

[2] Design a finite automaton to model a traffic light control system at a road intersection. The automaton should represent different states corresponding to the traffic lights, such as Red, Green, and Yellow. Define the transitions between these states based on a timer or control signals, ensuring that the sequence follows the standard order (Red → Green → Yellow → Red).

Include considerations for timing constraints, such as how long each light remains active, and describe how the automaton handles continuous cycling of signals. Additionally, explain how this model can be extended to handle pedestrian crossing signals or emergency vehicle overrides, ensuring safe and efficient traffic flow.

Concept

A traffic light is a deterministic finite-state controller. Each light colour is a state, and the input symbol T is the timer-expiry signal. Because the lights must change in a fixed order, every state has exactly one successor, which makes the machine a DFA with a cyclic structure.

Formal Definition

- Q = {Red, Green, Yellow}
- Σ = {T} (T = timer expired for the current light)
- Start state = Red
- F = ∅ (a controller runs forever, so it has no accepting state; every state is "valid")
- δ(Red,T) = Green, δ(Green,T) = Yellow, δ(Yellow,T) = Red

Current state	Input	Next state	Duration (timing constraint)
Red	T	Green	Red stays on for 30 s
Green	T	Yellow	Green stays on for 25 s
Yellow	T	Red	Yellow stays on for 5 s

Transition Diagram

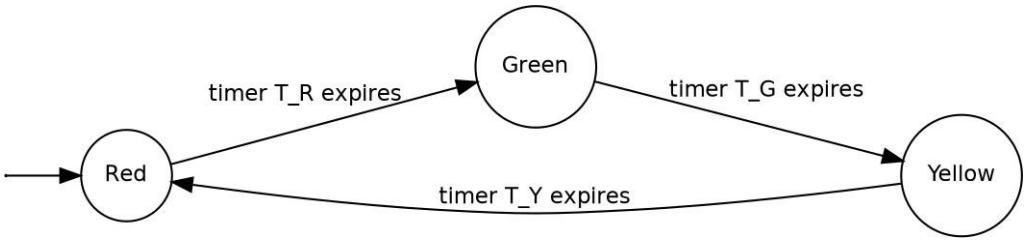


Fig 2.1 Basic traffic light automaton

Timing and Continuous Cycling

- Each state has a timer. A state is held until its timer expires; the expiry produces the input T and fires the transition. The durations (30 s, 25 s, 5 s) are example values and are set by the controller; a full cycle takes 30 + 25 + 5 = 60 s.
- If a finite automaton must count time itself, each light is split into sub-states $R_1 \ldots R_{30}$, $G_1 \ldots G_{25}$, $Y_1 \ldots Y_5$ and the input is a 1-second tick; the last sub-state of each light moves to the next light.
- Continuous cycling: the transition Yellow → Red returns the machine to the start state, so the sequence Red → Green → Yellow → Red repeats indefinitely. The input string is $(T\ T\ T)^*$.
- Safety: Green never goes directly to Red; Yellow is always inserted, and no state has two outgoing transitions on the same input, so two conflicting greens can never occur.

Simulation (input T T T T)

Step	Input	State after the move
0	—	Red
1	T	Green
2	T	Yellow
3	T	Red (cycle complete)
4	T	Green (cycle repeats)

Extension: Pedestrian Crossing and Emergency Override

The model is extended by adding states and input symbols while keeping the same cyclic structure:

- **Pedestrian crossing:** add the input B (pedestrian button pressed) and the state Ped Walk (all vehicle lights red, walk signal on). During Green, input B moves the machine to Ped Walk, then to Yellow/Red on timer T so the normal order is resumed. The pedestrian signal is the "walk/don't walk" output attached to the states Red and Ped Walk.
- **Emergency vehicle override:** add the input E (emergency vehicle detected) and the state All Red. From every state, E leads to All Red (all directions stopped, or green for the emergency direction only). The input C (emergency cleared) returns the machine to Red and normal cycling restarts.
- Safety and efficiency: the override has the highest priority (it is defined from every state), the pedestrian phase is entered only after the vehicle phase finishes, and all other inputs are ignored while in All Red.

Current state	T	B	E	C
Red	Green	—	All Red	—
Green	Yellow	Ped Walk	All Red	—
Yellow	Red	—	All Red	—
Ped Walk	Yellow	—	All Red	—
All Red	—	—	All Red	Red

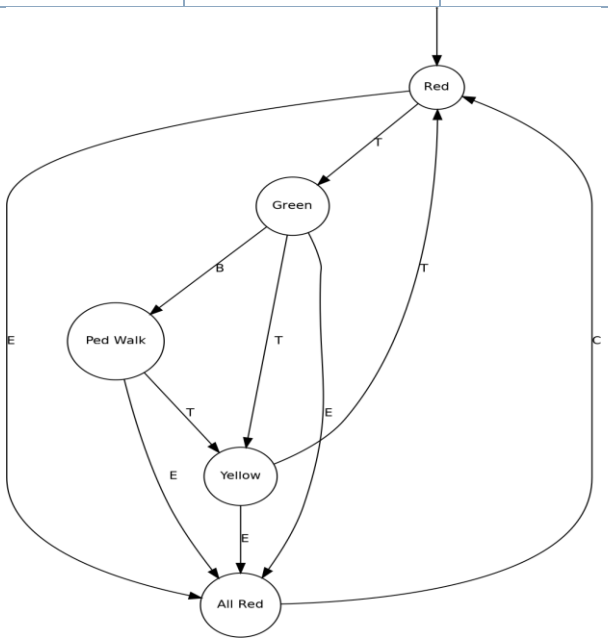


Fig 2.2 Extended automaton with pedestrian (B) and emergency (E, C) signals

Result: The 3-state cyclic DFA models the standard order Red → Green → Yellow → Red with timers, and the extension adds pedestrian and emergency handling safely.

[3] Design DFA using simulator to accept the string the end with ab over set {a,b)

W= aaabab

Concept

The DFA must remember how much of the suffix "ab" has just been read: nothing useful (q_0), the last symbol is a (q_1), or the last two symbols are ab (q_2, accepting). The language is L = { w ∈ {a,b}* | w ends with ab }.

Formal Definition

- Q = {q_0, q_1, q_2}, Σ = {a, b}
- Start state = q_0, F = {q_2}
- q_0 = no progress, q_1 = last symbol read is a, q_2 = last two symbols read are ab

State	a	b
→ q0	q1	q0
q1	q1	q2
* q2	q1	q0

Transition Diagram (as drawn in the simulator)

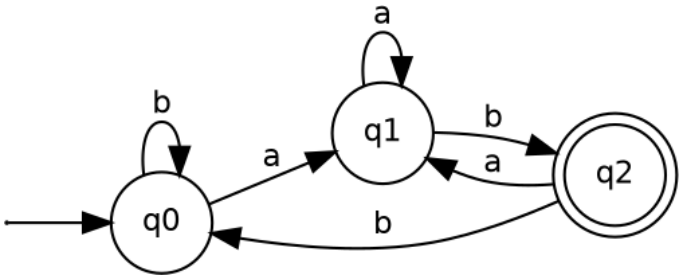


Fig 3 DFA accepting strings that end with ab

Steps in the Simulator (Autosim / JFLAP)

- Create three states q0, q1, q2; mark q0 as the initial state and q2 as the final state.
- Add the transitions: q0–a→q1, q0–b→q0, q1–a→q1, q1–b→q2, q2–a→q1, q2–b→q0.
- Run the input string aaabab and step through it.

Simulation of W = aaabab

Step	Symbol read	Current state	Next state
1	a	q0	q1
2	a	q1	q1

3	a	q1	q1
4	b	q1	q2
5	a	q2	q1
6	b	q1	q2

δ(q0,aaabab): q0 → q1 → q1 → q1 → q2 → q1 → q2.

Result: After reading all 6 symbols the DFA is in q2 ∈ F, so the string W = aaabab is **ACCEPTED**

(it ends with ab).

4 Design a Deterministic Finite Automaton (DFA) using a simulator to accept only the input strings "c", "bc", and "bcaaa" over the alphabet {a, b, c}. The automaton should begin from an initial state and transition through intermediate states based on the sequence of input symbols, ensuring that each valid string follows a unique path. For example, the string "c" should be accepted directly from the start state, while "bc" should pass through states representing "b" and then "c". Similarly, "bcaaa" should be processed through a sequence of states that track each additional "a" after "bc". Any deviation from these exact patterns should lead to a dead state, ensuring that no other strings are accepted. Clearly define the states, transition functions, start state, and accepting states, and implement the DFA in a simulator such as Autosim to verify its correctness.

Concept

The language is finite, L = {c, bc, bcaaa}. A finite language is recognised by a DFA shaped like a tree: one path per string, and a dead state that traps every other input. Each state stores the exact prefix read so far.

Formal Definition

- $Q = \{q_0, q_1, q_2, q_3, q_4, q_5, q_6, \text{dead}\}$, $\Sigma = \{a, b, c\}$
- Start state = q_0
- Accepting states $F = \{q_2, q_3, q_6\}$

State	Meaning (prefix read)	a	b	c
→ q0	ε (start)	dead	q1	q2
q1	b	dead	dead	q3
* q2	c	dead	dead	dead
* q3	bc	q4	dead	dead
q4	bca	q5	dead	dead
q5	bcaa	q6	dead	dead
* q6	bcaaa	dead	dead	dead
dead	any other input	dead	dead	dead

Transition Diagram

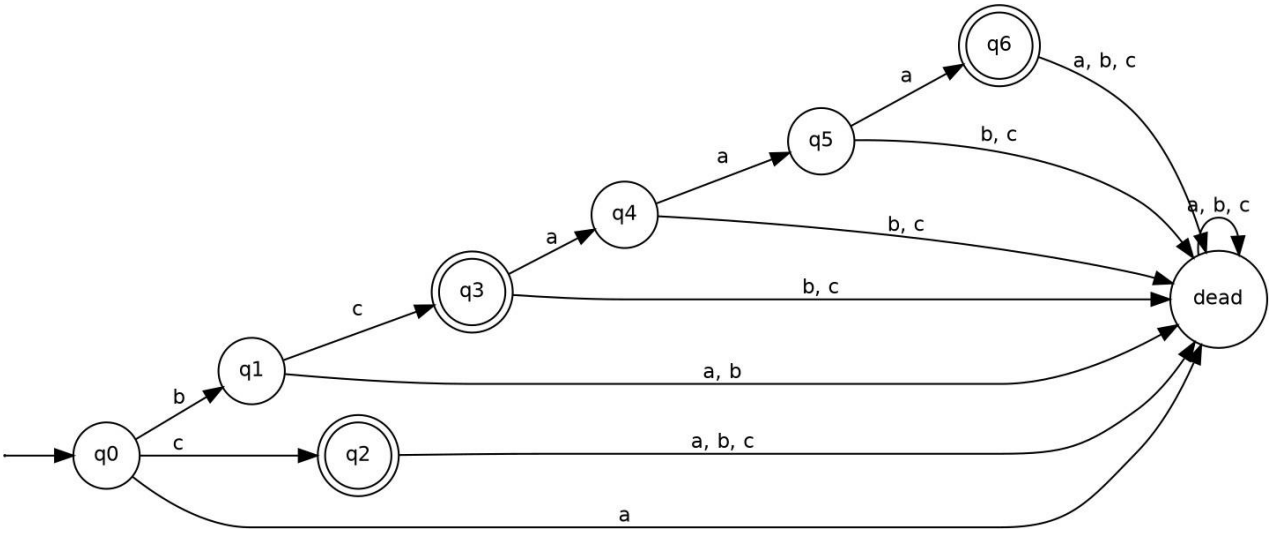


Fig 4 DFA accepting only c, bc and bcaaa

Steps in the Simulator (Autosim / JFLAP)

- Create states q0–q6 and a dead state; q0 is initial; q2, q3 and q6 are final.
- Enter the transitions from the table above; send all missing symbols to dead and make dead loop on a, b, c.
- Run each test string and compare with the expected result.

Verification

Input string	Path followed	Result
c	q0 → q2	Accepted
bc	q0 → q1 → q3	Accepted
bcaaa	q0 → q1 → q3 → q4 → q5 → q6	Accepted
ε (empty)	q0	Rejected (q0 ∉ F)
b	q0 → q1	Rejected
bca	q0 → q1 → q3 → q4	Rejected
bcaaaa	… → q6 → dead	Rejected
cc	q0 → q2 → dead	Rejected
ab	q0 → dead	Rejected

Result: The 8-state DFA (including the dead state) accepts exactly {c, bc, bcaaa} and rejects every other string.

[5] Design a Deterministic Finite Automaton (DFA) using a simulator to accept all strings over the alphabet {a, b} that begin with either "a" or "b". The automaton should start in an initial state and immediately transition to an accepting state upon reading the first input symbol, since any valid string must start with either "a" or "b". After the first symbol is processed, the DFA should remain in an accepting state regardless of the subsequent sequence of "a" and "b", thereby allowing any combination of characters to follow. Additionally, include a dead state to handle invalid or empty inputs if required by the simulator. Clearly define the states, transition rules, start state, and accepting states, and verify the design using a simulator such as Autosim to ensure that all valid strings are accepted and invalid ones are rejected.

Concept

Over $\Sigma = \{a, b\}$ every non-empty string begins with a or b, so the language is $L = \{ w \in \{a,b\}^* \mid |w| \geq 1 \} = (a + b)(a + b)^*$. Only two pieces of information matter: "nothing read yet" and "at least one symbol read".

Formal Definition

- $Q = \{q_0, q_1\}$, $\Sigma = \{a, b\}$
- Start state = q_0 (non-accepting, so the empty string is rejected)
- Accepting states $F = \{q_1\}$
- Dead state: not required here, because both a and b are defined in every state and the empty string is already rejected by q_0. If the simulator insists on a dead state (for symbols outside {a, b}), add one non-accepting state that loops on every symbol; no valid string ever reaches it.

State	a	b
→ q0	q1	q1
* q1	q1	q1

Transition Diagram

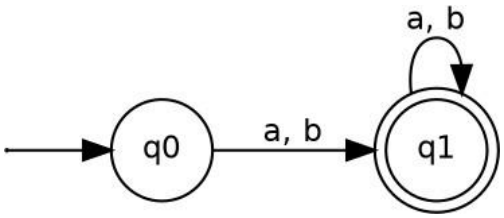


Fig 5 DFA accepting all strings that begin with a or b

Steps in the Simulator (Autosim / JFLAP)

- Create q0 (initial) and q1 (final).

- Add q0–a,b→q1 and the self-loop q1–a,b→q1.
- Run the test strings below.

Verification

Input string	Path followed	Result
a	q0 → q1	Accepted
b	q0 → q1	Accepted
ab	q0 → q1 → q1	Accepted
bba	q0 → q1 → q1 → q1	Accepted
abababb	q0 → q1 → … → q1	Accepted
ε (empty)	q0	Rejected (q0 ∉ F)

Result: The 2-state DFA accepts every string that starts with a or b (all non-empty strings over

{a, b}) and rejects the empty input.